

Things we'll need to look at

- Why does this work at all?
In other words, why is $m \equiv c^d \equiv m^{de} \pmod{n}$?
- Why should this be secure?
- How do we find the large, random primes needed for the key generation step?

We'll eventually address all of these questions!

Practical comments:

- For demonstration purposes, it will be convenient to operate on strings over some alphabet Σ rather than on integers in $\mathbb{Z}_n$.

- There are various ways of encoding strings as numbers and vice versa.
- Since $n \gg |\Sigma|$, our previous method is no good.
- The following method will be used for Assignment 3.

Let $B = |\Sigma| = \text{no. of symbols in our alphabet } \Sigma$.

We identify $\Sigma = \{0, \dots, B-1\}$.

A string of length $k+1$ over Σ then corresponds to a sequence
 $(a_0, \dots, a_k)$

of integers a_i with $0 \leq a_i < B$.

To this sequence, we attach the integer

$$A = a_0 + a_1 B + a_2 B^2 + \dots + a_k B^k$$

70

Exercise: $A \leq B^{k+1} - 1$ with equality iff $a_0 = \dots = a_k = B - 1$.

Hence: given any $n \geq 1$ (e.g. the modulus of an RSA key),

if $(\text{length of string}) = k+1 \leq \lfloor \log_B(n) \rfloor$,

then $A < n$.

We can thus use integers between 0 and $n-1$ to represent certain strings over Σ , including all strings of length $\leq \lfloor \log_B(n) \rfloor$.

However: this correspondence is not 1-1 since

$(a_0, \dots, a_k)$

and

$(a_0, \dots, a_k, 0, \dots, 0)$

any number of zeros

correspond to the same number.

Some possible ways around this ambiguity

- Pad all strings to the same length.

We could, for instance, add a "dummy character" which will be removed after decryption.

- Treat the character from our alphabet corresponding to the number 0 differently in that trailing copies of this character are silently removed from strings.

(For Assignment 3, we'll do this with 0 = space.)